Lab 6: Model Evaluation & Model Artifacts

- **Course:** Engineering of Intelligent Models
- **Module:** M3. Model Orchestration & Automation
- **Focus:** Train/Test Splitting, Artifact Registry, Visual Forecasting, and MLflow Integration
- **Branch:** `Lab6`

1. Goal of the Laboratory

Training a model that performs well on historical data is trivial; training a model that generalizes to the future is the true challenge. In this final laboratory of the lifecycle series, we will establish a strict **Evaluation Pipeline**.

Because we have vastly different architectures (Multivariate Deep Learning vs. Univariate Statistical), we must establish a **Primary Business Metric**. We will evaluate all models strictly on their ability to predict our primary target: **Temperature (`temperature_2m`)**.

To simulate a real-world scenario, we will acquire a larger dataset, strictly reserve the last 30 days (720 hours) as an unseen test set, and physically save our trained models as **Artifacts** in MLflow. Our evaluation script will then load these artifacts from the registry, generate predictions on the holdout data, compute our primary business metric (Temperature RMSE/MAE), and log a visual forecast plot directly to the MLflow server.

2. Version Control & Data Acquisition

Before writing any code, we must secure our project history and acquire enough data to train a robust time-series model.

Step 1: Branching Strategy

Save your Lab 5 progress and create a new isolated environment for Lab 6:

In [ ]:
```
!git add .
!git commit -m "feat: Lab5 --> Notebook 2 completed."
!git checkout -b Lab6
```

Step 2: Acquire Historical Data

We need a substantial dataset. Open your Apache Airflow UI (`http://localhost:8080`), navigate to the `daily_weather_ingestion` DAG, and trigger a **Single Run** with the following parameters:

- `start_date`: `"2025-01-01"`
- `end_date`: `"2025-12-31"`

Once the DAG succeeds, your Python script will seamlessly append this entire year of data to your existing CSVs.

3. Preventing Data Leakage (Updating the Training Script)

Currently, our `train_model.py` ingests the entire CSV. We must update it to hold back the last 720 hours (30 days * 24 hours) from the training process. Furthermore, we must command MLflow to save the actual model files (`.onnx`, `.pt` or Prophet binaries) as artifacts.

Open `src/training/train_model.py` and implement the Train/Test split and Artifact Logging. Look for the `FIX` comments in the code to identify the necessary changes.

```
# ... (previous imports and setup) ...
# FIX 0: Import MLflow logging features compatible with both Prophet and PyTorch
# Add Prophet and PyTorch MLflow imports at the end of your import section
from mlflow.prophet import log_model as log_prophet_model
from mlflow.pytorch import log_model as log_pytorch_model

# ... inside your train() function ...
data_path = "data/raw/historical_weather-Lisbon.csv"
df = pd.read_csv(data_path)
```

```
# FIX 1: Strict Train/Test Split to prevent Data Leakage
# Reserve the last 720 rows (30 days) for the Evaluation script
train_df = df.iloc[:-720].reset_index(drop=True)

current_time = datetime.now().strftime("%Y%m%d_%H%M%S")
dynamic_run_name = f"{cfg.model.name}_Training_Run_{current_time}"

with mlflow.start_run(run_name=dynamic_run_name) as run:
    mlflow.log_params(OmegaConf.to_container(cfg.model, resolve=True))
    mlflow.log_param("data_dvc_hash", get_dvc_hash("data/raw.dvc"))
    mlflow.log_param("test_set_reserved_hours", 720) # Document the split

    if cfg.model.name in ["LSTM", "GRU"]:
        # Use train_df instead of df
        features = train_df[['temperature_2m', 'relative_humidity_2m',
'precipitation']].values
        X, y = create_sliding_windows(features, cfg.model.sequence_length)
        dataset = TensorDataset(X, y)

        dataloader = DataLoader(dataset, batch_size=cfg.model.batch_size, shuffle=False,
num_workers=3)
        model = WeatherLSTM(cfg.model) if cfg.model.name == "LSTM" else WeatherGRU(cfg.model)

        mlf_logger = MLFlowLogger(experiment_name="Weather_Forecasting_Models",
tracking_uri="http://mlflow_server:5000", run_id=run.info.run_id)

        trainer = pl.Trainer(max_epochs=cfg.model.epochs, logger=mlf_logger,
enable_checkpointing=False, log_every_n_steps=5)
        trainer.fit(model, dataloader)

        # FIX 2.1: Grab one batch of data to serve as the input example (required due to
export_model=True)
        example_input, _ = next(iter(dataloader))
        input_example = example_input.numpy()

        # FIX 2.2: Save the PyTorch model artifact to MLflow
        log_pytorch_model(model, name="model_artifact", export_model=True, code_paths=
["src/training/models"], input_example=input_example)

    elif cfg.model.name == "Prophet":
        prophet_model = WeatherProphet(cfg.model)
        fitted_model = prophet_model.fit(train_df, target_column='temperature_2m') # Use
train_df

        # (Loss calculation code remains the same, but uses train_df) ...

        # FIX 2: Save the Prophet model artifact to MLflow
        log_prophet_model(fitted_model, name="model_artifact")
        logger.info("Prophet fitting and artifact logging complete.")
```

Code Explanation:

- `train_df = df.iloc[:-720].reset_index(drop=True)` : This line creates a new DataFrame `train_df` that excludes the last 720 rows, ensuring that our training process only sees historical data up to that point.
- `log_pytorch_model` and `log_prophet_model` : These functions save the trained model files as artifacts in MLflow, allowing us to retrieve them later for evaluation. The artifact will be stored under the name "model_artifact" in the MLflow run.
  - The `export_model=True` argument for PyTorch ensures that the model is saved in a safe format (via `torch.export.save`). This saving format exports the model as a traced graph. This format addresses the security vulnerability of `CloudPickle` format, avoiding the warning logged by MLflow.
  - Also, due to the export format, we need to provide an `input_example` which is a sample input tensor that matches the expected input shape of the model. This allows MLflow to understand how to load and use the model later during evaluation. This is mandatory when using `export_model=True` to ensure the model can be properly loaded and used in the future without needing the original codebase.

4. The Continuous Evaluation & Artifact Script

Now, we will build a dedicated script that acts as our daily monitoring agent. It does not train anything; it simply queries MLflow for the latest registered model artifact, loads the most recent 720 hours (30 days) of ingested data, and calculates the error. Because our data ingestion DAG runs daily, this 720-hour window shifts forward every 24 hours, giving us a moving window of ground truth.

Replace the old placeholder in `src/evaluation/evaluate_model.py` with the following code:

> *NOTE: Ensure `plotly` is in your `requirements.txt`*. After that, don't forget to rebuild your Docker image and restart your Docker containers to apply the new dependencies and code changes.

```
# src/evaluation/evaluate_model.py
import hydra
from PIL import Image
from omegaconf import DictConfig
import mlflow
import pandas as pd
import numpy as np
import torch
import plotly.graph_objects as go
import logging
import os
import time
import io
from datetime import datetime

from mlflow.pytorch import load_model as load_pytorch_model
from mlflow.prophet import load_model as load_prophet_model

logger = logging.getLogger(__name__)


def calculate_metrics(y_true, y_pred):
    y_true, y_pred = np.array(y_true).flatten(), np.array(y_pred).flatten()
    return np.sqrt(np.mean((y_true - y_pred) ** 2)), np.mean(np.abs(y_true - y_pred))


@hydra.main(version_base=None, config_path="../../conf", config_name="config")
def evaluate(cfg: DictConfig):
    logger.info(f"--- Starting Daily Evaluation Pipeline for {cfg.model.name} ---")

    mlflow.set_tracking_uri("http://mlflow_server:5000")
    client = mlflow.tracking.MlflowClient()
    experiment = client.get_experiment_by_name("Weather_Forecasting_Models")

    # 1. Load the Shifting 30-day Test Set
    # Because ingestion runs daily, the tail of this CSV contains new unseen data every day!
    df = pd.read_csv("data/raw/historical_weather-Lisbon.csv")
    test_df = df.iloc[-720:].reset_index(drop=True)
    dates = pd.to_datetime(test_df['date'])
    y_true = test_df['temperature_2m'].values

    # 2. Find the latest trained run to evaluate
    query = f"tags.mlflow.runName LIKE '{cfg.model.name}_Training_Run_%'"
    runs = client.search_runs(experiment_ids=[experiment.experiment_id], filter_string=query,
order_by=["start_time DESC"], max_results=1)

    if not runs:
        logger.error(f"No trained model found for {cfg.model.name}. Skipping evaluation.")
        return

    latest_run_id = runs[0].info.run_id

    # 3. Load the Model Artifact directly from MLflow Registry
    artifact_uri = f"runs:/{latest_run_id}/model_artifact"
    logger.info(f"Evaluating Model Artifact: {artifact_uri}")
```

```
    # Re-open the EXISTING training run to append today's metrics
    with mlflow.start_run(run_id=latest_run_id):
        if cfg.model.name in ["LSTM", "GRU"]:
            loaded_model = load_pytorch_model(artifact_uri)

            features = test_df[['temperature_2m', 'relative_humidity_2m',
'precipitation']].values
            seq_length = cfg.model.sequence_length

            y_pred = []
            with torch.no_grad():
                for i in range(len(features) - seq_length):
                    x_input = torch.tensor(np.array([features[i:(i + seq_length)]]),
dtype=torch.float32)
                    pred = loaded_model(x_input)
                    y_pred.append(pred.numpy()[0][0])

            y_true_aligned = y_true[seq_length:]
            dates_aligned = dates[seq_length:]

        elif cfg.model.name == "Prophet":
            loaded_model = load_prophet_model(artifact_uri)
            prophet_test_df = pd.DataFrame({'ds': dates})
            forecast = loaded_model.predict(prophet_test_df)
            y_pred = forecast['yhat'].values

            y_true_aligned = y_true
            dates_aligned = dates

        # 4. Calculate and Log Metrics
        # By logging to the same run_id every day, MLflow builds a historical drift chart!
        rmse, mae = calculate_metrics(y_true_aligned, y_pred)

        # Use a timestamp as the step so MLflow tracks the progression over time
        current_time = int(datetime.now().strftime("%Y%m%d"))
        mlflow.log_metric("daily_test_rmse", float(rmse), step=current_time)
        mlflow.log_metric("daily_test_mae", float(mae), step=current_time)

        # 5. Generate and Log Forecast Plot
        fig = go.Figure()
        fig.add_trace(go.Scatter(x=dates_aligned, y=y_true_aligned, mode='lines', name='Actual
Temp', line=dict(color='blue')))
        fig.add_trace(go.Scatter(x=dates_aligned, y=y_pred, mode='lines',
name=f'{cfg.model.name} Forecast', line=dict(color='red', dash='dash')))
        fig.update_layout(title=f"{cfg.model.name} - 30 Day Forecast vs Actuals",
xaxis_title="Date", yaxis_title="Temperature (°C)")

        current_time = datetime.now().strftime("%Y%m%d_%H%M%S")
        plot_filename = f"forecast_plot_{cfg.model.name}_{current_time}.html"  # HTML natively
supported by MLflow UI
        mlflow.log_figure(fig, plot_filename)

        logger.info(f"Daily Evaluation Complete! Test RMSE: {rmse:.4f} | Test MAE: {mae:.4f}")


if __name__ == "__main__":
    evaluate()
```

Code Explanation:

- `calculate_metrics` : A helper function that computes RMSE and MAE between the true and predicted values. This function is used to evaluate the performance of the model on the test set.
- `client = mlflow.tracking.MlflowClient()` : Initializes the MLflow client to interact with the MLflow tracking server. This allows us to query for existing runs and log new metrics and artifacts. We use this client to search for the latest training run corresponding to the model we want to evaluate.
- `experiment = client.get_experiment_by_name("Weather_Forecasting_Models")` : Retrieves the MLflow experiment object for our weather forecasting models. This is necessary to search for runs within this

specific experiment.

- `test_df = df.iloc[-720:].reset_index(drop=True)` : This line extracts the last 720 rows of the dataset, which represent the most recent 30 days of data. This subset will be used as the test set for evaluation.
- `query = f"tags.mlflow.runName LIKE '{cfg.model.name}_Training_Run_%'"` : Constructs a query string to search for MLflow runs that match the naming pattern of our training runs. This allows us to dynamically find the latest run for the specific model architecture we want to evaluate.
- `runs = client.search_runs(...)` : Executes the search query against the MLflow tracking server to retrieve a list of runs that match the criteria. We order the results by start time in descending order to get the most recent run at the top.
- `artifact_uri = f"runs:/{latest_run_id}/model_artifact"` : Constructs the URI to access the model artifact stored in MLflow for the latest training run. This URI is used to load the model for evaluation.
- `loaded_model = load_pytorch_model(artifact_uri)` and `loaded_model = load_prophet_model(artifact_uri)` : Depending on the model type, we use the appropriate MLflow function to load the model artifact directly from the MLflow (since we logged it as an artifact in the training script). This allows us to evaluate the exact model that was trained without needing to access the original codebase or file system.
- `x_input = torch.tensor(np.array([features[i:(i + seq_length)]]), dtype=torch.float32)` : Prepares the input data for the PyTorch model by creating sliding windows of the test features. This is necessary to generate predictions in the same format that the model was trained on.
- `mlflow.log_metric(...)` : Logs the calculated RMSE and MAE metrics to the same MLflow run that was created during training. By using the same `run_id` , we can track the performance of the model over time as new evaluation metrics are logged daily. This creates a historical record of the model's performance, allowing us to visualize trends and detect potential data drift in the MLflow UI.
- `mlflow.log_figure(fig, plot_filename)` : Logs the generated Plotly figure as an artifact in MLflow. This allows us to visualize the forecast vs actuals directly in the MLflow UI, providing a visual representation of the model's performance on the test set. The plot is saved as an HTML file, which is natively supported by MLflow for interactive viewing.

5. Orchestrating Continuous Monitoring (Apache Airflow)

We now create a brand new DAG dedicated exclusively to evaluating our models. This enforces the ultimate separation of concerns:

- `daily_weather_ingestion` : Runs `@daily` to get new ground truth.
- `monthly_model_training` : Runs `@monthly` to train a fresh model.
- `daily_model_evaluation` : Runs `@daily` to test the monthly model against the daily data.

Step 1: Remove the Evaluation Logic from the Training DAG (if still present)

Open your `dags/model_training_dag.py` and ensure the `evaluate_model.py` bash command is completely **removed** from the training logic. The training DAG should now *only* train and save the artifact.

Step 2: Create `dags/model_evaluation_dag.py` .

We will use the same dynamic folder-scanning trick to evaluate all available models.

```
# dags/model_evaluation_dag.py
import os
from airflow import DAG
from airflow.providers.standard.operators.bash import BashOperator
from datetime import datetime, timedelta

# Dynamically scan for available models
CONF_MODEL_DIR = "/opt/airflow/conf/model"
try:
    available_models = [f.replace('.yaml', '') for f in os.listdir(CONF_MODEL_DIR) if
f.endswith('.yaml')]
except FileNotFoundError:
    available_models = ["lstm", "gru", "prophet"]

# Default arguments applied to all tasks in the DAG
default_args = {
    'owner': 'mlops_engineer',
```

```
    'depends_on_past': False,
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 5,
    'retry_delay': timedelta(seconds=5),
}

# Define the Evaluation DAG to run DAILY
with DAG(
        'daily_model_evaluation',
        default_args=default_args,
        description='Evaluates active ML models daily',
        schedule='@daily',
        catchup=False,
        tags=['weather_capstone', 'evaluation'],
) as dag:
    # Dynamically generate an evaluation task for EVERY model
    for model_name in available_models:
        evaluate_model_task = BashOperator(
            task_id=f'evaluate_{model_name}_model',
            bash_command=f'cd /opt/airflow && python src/evaluation/evaluate_model.py model=
{model_name}'
        )
```

6. Executing the Continuous Training & Evaluation Pipelines

You have officially constructed a professional, enterprise-grade Continuous Training and Evaluation pipeline! ⛏️

Your Apache Airflow DAG (`model_training_dag.py`) is already configured from Lab 5 to execute `train_model.py` script.

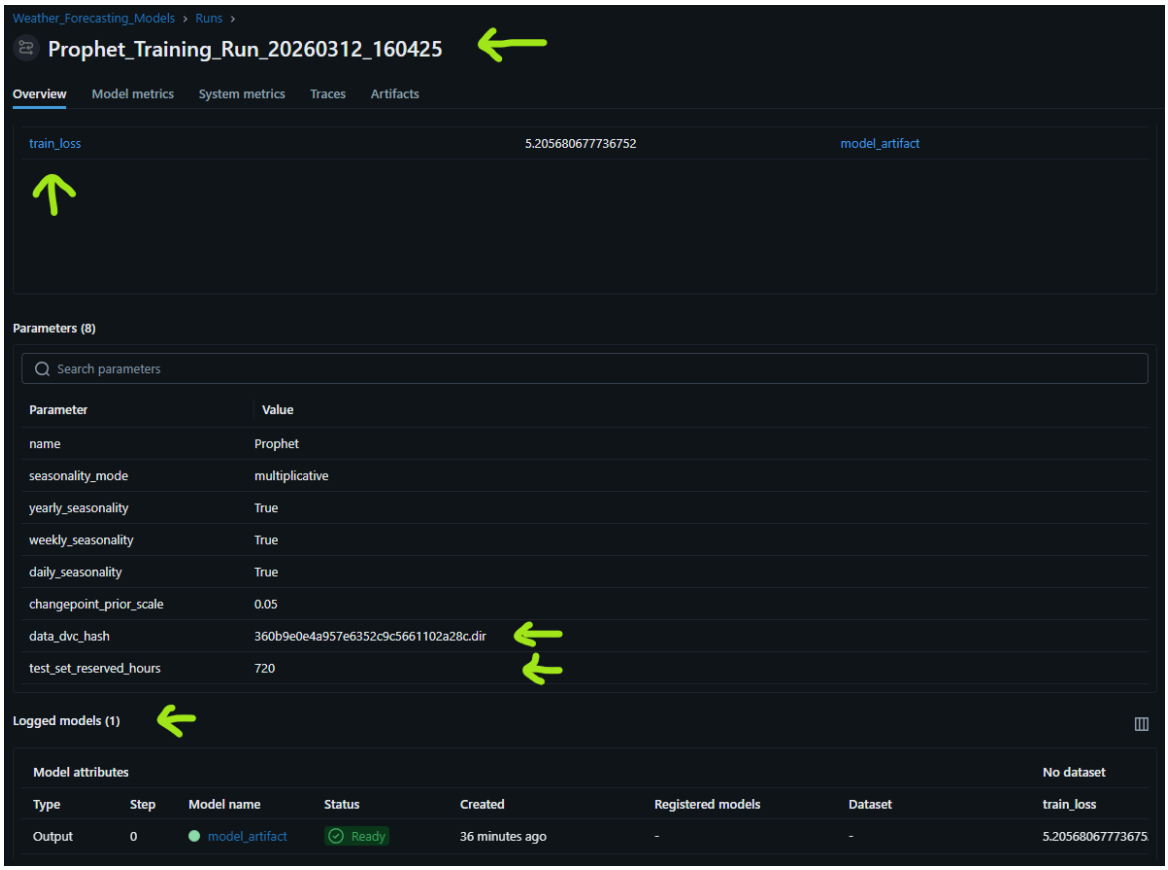
6.1 Check Model Artifact Logging:

1. Open Airflow and trigger the `monthly_model_training` DAG, selecting `"all"` models.
2. Wait for the tasks to succeed (It will take longer because we are training on a full year of data now).
3. Open MLflow (`http://localhost:5000`).
4. Select the "Weather_Forecasting_Models" experiment. You can also toggle the Charts view to see the metrics whilst the training is still running (it updates the run in real-time), and group the runs by `name` to easily compare the different architectures.

5. Click on any run to see the details. For example, the more recent `Prophet_Training_Run_AAAAMMDD_HHMMSS` run. Inside, you will find:

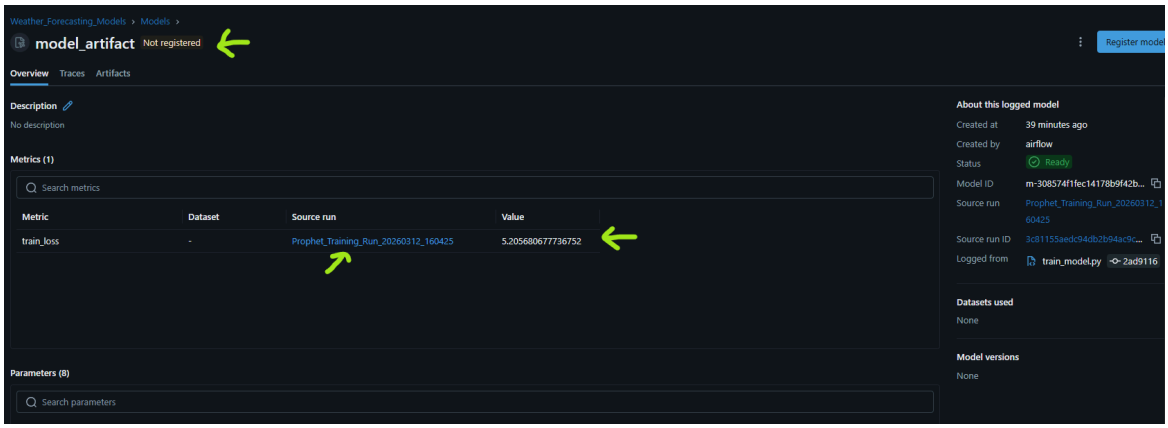
- **Parameters:** Your DVC Hash, Hydra configs, and Split logic.

- **Metrics:** Train Loss alongside Test RMSE and Test MAE.
- **Artifacts:** A `model_artifact/` folder containing your physical model files in:
  - Pytorch's Pickle format: `.pt` or `.pth`
  - Pytorch's Safetensors format: `.pt2` (if you have `export_model=True` in your training script, which is recommended for security reasons)
  - Prophet's format: `.pr`

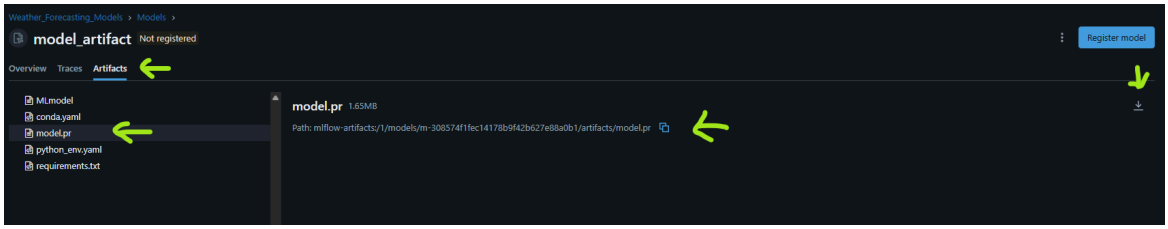


6. Now, if you click on the `model_artifact` folder (option inside `Logged Models`), you can :

- Check the associated source run and parameters that led to that artifact.
- Register that artifact in the MLflow Model Registry (which we will cover in Lab 7) to manage its lifecycle and deployment.

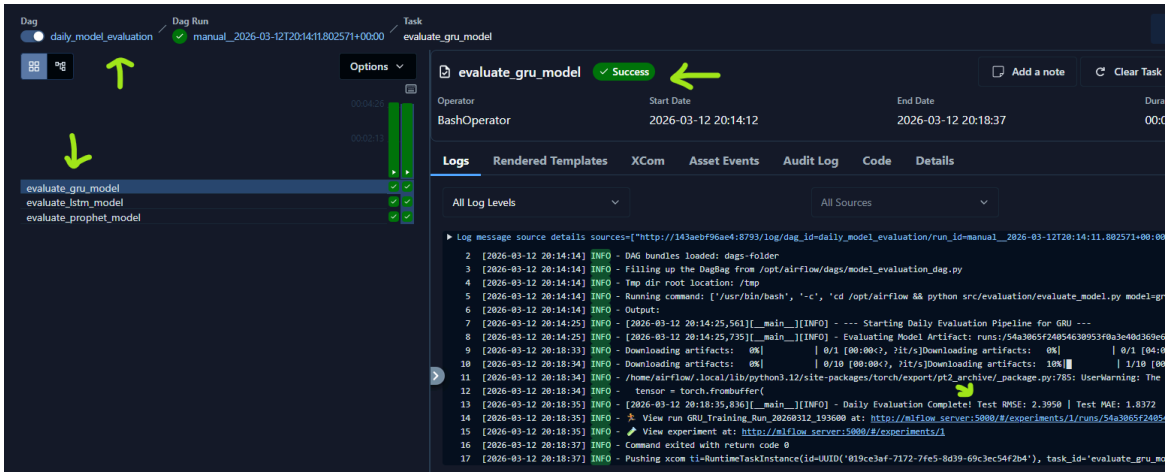


- And if you click on the `Artifacts` tab, you can download the actual model file to inspect its size, creation date, and even open it with the appropriate software (e.g., ONNX Viewer for `.onnx` files or PyTorch for `.pt` files).



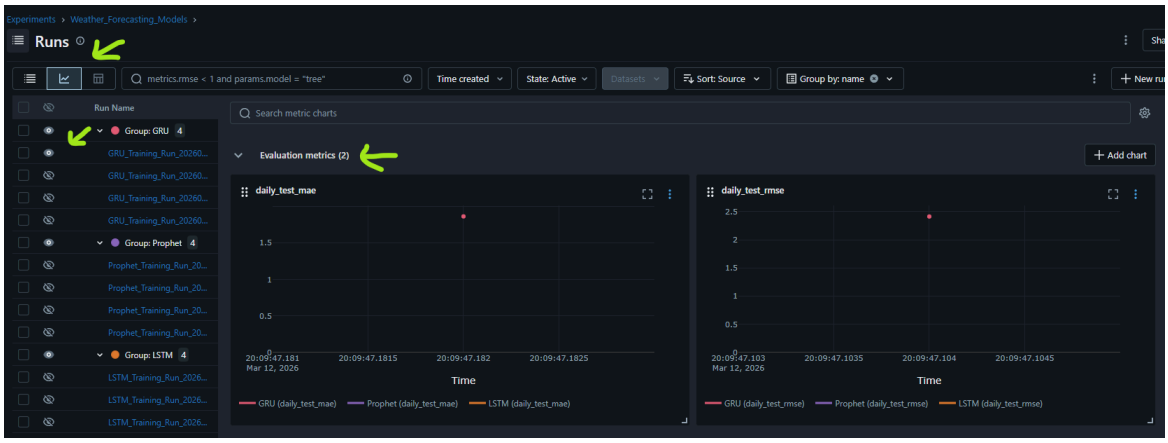
6.2 Check Continuous Evaluation:

1. Manually trigger your new `daily_model_evaluation` DAG.
2. Select a run. If everything is set up correctly, you should see the following in the Airflow logs of the evaluation task:



3. Open MLflow (`http://localhost:5000`), select a run, and click on the `daily_test_rmse` metric.

- Notice that I've logged the RMSE with a `step` corresponding to the current date (in `YYYYMMDD` format). This allows MLflow to plot the RMSE progression over time as new evaluation runs are logged daily.
- Also, I've grouped the runs by `name` (which corresponds to the model architecture) to easily compare the performance of different models over time. You can toggle the Charts view to visualize the RMSE trends for each model.
- I've also created a new section called 'Evaluation metrics' in the MLflow UI to separate the evaluation metrics from the training metrics, making it easier to analyze the model's performance on unseen data.
- And, if you hide the other runs, you can see the RMSE progression for a single model. This is the visual proof of how your model's performance evolves as it encounters new data every day.



Right now, it is a single point on a graph. But as your Airflow scheduler runs the ingestion and evaluation DAGs every night, that metric will begin to form a line. When winter turns to spring, or if global weather patterns shift, your students will see that line climb -> the undeniable visual proof of **Data Drift**!

4. Finally, if you click on a run, and then on the 'Artifacts' tab, you will find the forecast plot that was generated during evaluation. This plot visually compares the model's predictions against the actual temperature values for

the most recent 30 days of data. By inspecting this plot, you can gain insights into how well the model is capturing trends and patterns in the data, and identify any discrepancies or areas where the model may be underperforming.

> *NOTE: I've only trained for 15 epochs for testing purposes, so the RMSE is quite high. You can increase the number of epochs in your training config (Hydra YAML files) to see better performance, but be aware that it will take longer to train.*

7. Next Steps

The stage is now perfectly set for Lab 7, where we will register our models in the MLflow Model Registry, implement a manual approval workflow, and deploy our best model to a REST API using FastAPI. The evaluation pipeline we built here will serve as the critical monitoring agent that ensures our deployed model continues to perform well in production.